

**Integrating a Large Language Model into a Task Management Application: An
Exploratory Implementation**

Douglas Fulcher

Bellevue University

Applied Data Science DSC680 T301 - Fall 2025

Instructor: Amirfarrokh Iranitalab

Abstract

This project explores the integration of a large language model into a Python task management application. Using the Todoist API and Google's Gemini model, I built a Streamlit interface that lets users create and retrieve tasks through natural language. Rather than focusing on predictive modeling, I wanted to understand systems architecture and software engineering practices for AI applications. The project provided me with hands-on experience with modern Python tooling, including mypy for type safety, ruff for code quality, bandit for security scanning, and Podman for containerized deployment. The system works for its intended purpose, but there is still plenty of work to be done before it can go into production. This paper addresses the business problem, outlines my methods, discusses what I learned, highlights the challenges I encountered, identifies areas that are still missing, and suggests potential next steps.

Integrating a Large Language Model into a Task Management Application: An Exploratory Implementation

Business Problem Statement

People who work with information all day rely on task management platforms to stay organized and efficient. However, these systems require you to learn their specific interfaces and ways of doing things. I wanted to see if an LLM could enable people to describe their needs in plain English without requiring a redesign of the entire task management system.

I kept the scope narrow on purpose—just creating tasks from natural language and pulling up active tasks. This allowed me to dig deep into integration challenges, validation patterns, and deployment strategies, rather than spreading myself too thin. It is an exploratory approach, not a complete solution. I learned a lot about connecting LLMs to APIs, but turning this into something production-ready would take significantly more work.

Background Research

Traditional machine learning projects teach you how to train and evaluate models. However, working with LLMs is different—you are dealing with prompt engineering, parsing outputs, managing errors, and integrating services. The Todoist API gave me a real-world platform to explore these challenges. Other developers have written about combining LLMs with SaaS platforms to build productivity assistants, and I drew on that work.

I took an existing Streamlit application and modernized it to explore how object-oriented design, type safety, and containerization integrate with LLM functionality. My goal was to understand integration patterns, rather than building something complete. That educational focus shaped every decision I made.

System Architecture

The application sits between three things: the user, Gemini, and Todoist. It does two main jobs. First, it takes natural language from users and has the LLM interpret it to create structured task data that Todoist understands. Second, it fetches active tasks and shows them with links you can click.

I organized the code into separate layers. There is an LLM service that communicates with Gemini, a Todoist client that handles API calls, a Streamlit UI for users, and validation logic using Pydantic models. Keeping these components separate made development much easier because I could work on one piece without affecting the others.

Development Workflow and Tooling

I wanted to learn modern Python practices by actually using them. For configuration, I used `pyproject.toml` and `uv` for managing dependencies. Code quality was ensured by `ruff` (linting and formatting), `mypy` (type checking), and `bandit` (security scanning). Pre-commit

hooks run all these checks automatically, which showed me how to build quality gates into a development workflow.

The application runs in two modes: locally for quick development and in a Podman container to simulate deployment. I wrote tests for the main features; however, comprehensive testing would require additional investment. GitHub Copilot helped with scaffolding code, which was interesting for seeing how human-AI collaboration works in software development. I worked diligently to guide the AI using instruction files, chat modes, prompt files, and AGENTS.md, which are emerging methods of applying control to AI-assisted development. Sometimes it was helpful, sometimes the AI would go off on a tangent, and I had to revert to a prior branch and start again. Given that my job and emerging job postings for software engineers and data scientists alike increasingly require familiarity with these tools, I felt this was a suitable project to explore these concepts.

Data Flow and Validation

This project differs from typical modeling projects in that it does not utilize training datasets. Instead, data transforms as it moves through the system: user input becomes LLM prompts, LLM outputs are parsed into Pydantic models, and validated models are mapped to Todoist API payloads. I set up a dictionary that defines how LLM outputs (title, description, due date) align with what the API expects.

The validation layer was one of the most significant areas of learning. How do you safely connect probabilistic LLM outputs with deterministic API requirements? That question drove many of my design decisions. For instance, I had to create a mode adapter in the program, which assisted the AI in responding more intelligently to specific user prompts, such as saying hello or good morning. I also decided to error out rather than attempt to allow interaction when the user asked questions that were not task-related, such as “What color is a banana?” Allowing a more robust chat experience proved to detract from the agent’s ability to handle tasks adequately, so I attempted to control for this by limiting those interactions.

Results

The implementation does what I set out to do. You can create tasks in natural language and retrieve them, both locally and in containers. Type safety checks pass in both mypy (static analysis) and Pydantic (runtime validation), which proves you can maintain type guarantees when connecting LLMs to external APIs. Gemini interprets task requests well, and the system reliably sends validated data to Todoist. Running it in Podman showed me how containerized deployment provides consistent execution environments.

However, that is where the good news ends in terms of completeness. You cannot mark tasks complete, modify them, or filter them through the interface. Error handling covers common cases but needs expansion. Security includes environment-based credential management and output validation; however, comprehensive hardening is not implemented. There is a long list of things that would need to be built before this can be considered production-ready.

Discussion

Working on this taught me several things about LLM integration. Separating concerns across layers (LLM service, API client, validation, UI) made development and testing simpler. Each component can evolve independently, which is crucial when working with rapidly changing technologies.

Input and output validation is not optional—it is essential. The Pydantic validation boundary stops malformed data from propagating through the system. Without it, bad LLM outputs could cause API errors or worse. Modern Python tooling made a real difference, too. Uv speeds up environment management considerably, pre-commit hooks catch issues before they become problems, and type checking finds bugs before runtime.

Containerization proved valuable even for an exploratory project. Environmental consistency matters, and learning to think about deployment early pays off in the long run. From an educational standpoint, this project accomplished what I wanted. I gained practical experience with API integration, type safety patterns, containerization, and modern Python workflows—all skills that directly transfer to industry work. The limited scope allowed me to delve deeply into DevOps concepts, which meant thoroughly exploring core integration challenges without getting overwhelmed. However, the tradeoff was a lack of depth in features.

Challenges

Several technical challenges came up during development. Maintaining type safety across dynamic LLM interfaces required careful use of runtime checks to ensure consistency. You have to balance what static analysis can verify with practical realities.

Security and validation needed attention. LLM outputs must undergo rigorous checking before going to the API to prevent injection attacks or malformed data. Learning modern tooling standards (pyproject.toml, uv, containerization) meant picking up new patterns and configurations. Working with Copilot was useful, but it required oversight—you cannot simply trust everything it suggests. I addressed these through iteration, targeted testing, and documentation. It reinforced the importance of engineering discipline when working with AI systems.

Limitations

The current implementation has significant constraints. Features are restricted to task creation and listing. You cannot mark tasks complete, modify attributes, or apply filters. There is no offline testing, user acceptance testing, or telemetry to validate how it would work in real-world usage. I handle LLM output variability through parser validation rather than model fine-tuning or prompt optimization.

The single-user focus overlooks the need for multi-tenancy and access control, which production systems require. Error handling addresses common scenarios but lacks comprehensive coverage. These constraints reflect the exploratory nature of the project. They underscore the significant amount of work that remains before production deployment.

Future Uses/Applications

There are several directions this could go. Feature completeness would mean implementing task completion, modification, rescheduling, and filtering. Enhanced LLM interactions might include contextual assistance, workload summarization, or intelligent prioritization. Integration with other systems (calendars, email, CRM platforms) would show how flexible the architecture is.

Production readiness demands comprehensive error handling, extensive testing that includes edge cases and failure scenarios, robust logging and monitoring, authentication and authorization for multiple users, and performance optimization for scalability. Telemetry and feedback loops would enable continuous improvement. Each direction represents substantial additional learning about building reliable, maintainable LLM-enabled systems.

Recommendations

If you are doing something similar, start with modern Python tooling early. Use `pyproject.toml`, `uv`, `ruff`, `mypy`, and pre-commit hooks from the beginning to establish good habits and maintainable codebases. Design modular architectures that keep LLM interactions, business logic, and external API clients separate. This allows each piece to evolve and be tested independently.

Implement rigorous validation at boundaries between probabilistic AI and deterministic systems. Use typed models and explicit checks. Start with a narrow feature scope to focus on depth and reliability before expanding. Explore containerization to understand deployment consistency even in learning contexts. Document limitations transparently—it sets realistic expectations and guides future work. Recognize that moving from exploratory implementation to production deployment represents substantial additional investment in testing, security, monitoring, and operational robustness.

Ethical Assessment

Even exploratory implementations must consider their ethical implications. This system handles sensitive data, including task content and user credentials. I used environment-based configuration and encrypted API communication for secure storage and transmission. Users should understand the capabilities and limitations of LLMs. The system should clearly indicate when it cannot interpret requests, rather than making assumptions.

Deployment flexibility (local and containerized) supports fair access across different infrastructure capabilities. While this exploratory project addresses fundamental ethical concerns, the deployment of production would require a deeper consideration of privacy policies, data retention, consent mechanisms, and accessibility standards.

Conclusion

This project successfully explores LLM integration patterns through focused implementation. By constraining scope to task creation and retrieval, I could thoroughly examine architectural design, type safety, validation patterns, modern Python tooling, and

containerized deployment. The resulting system demonstrates that careful engineering can bridge the gap between probabilistic LLM outputs and deterministic API requirements, although significant additional work remains before achieving production readiness.

The educational value stems from a deep understanding of core integration challenges and contemporary development practices, rather than achieving completeness. As LLMs become standard components in business applications, the patterns explored here—modular architecture, rigorous validation, type safety, and operational tooling—provide essential foundations for building reliable intelligent systems. Future iterations can build upon this groundwork to expand features, enhance robustness, and explore additional integration scenarios.

Appendix A

Figures

Figure 1: Initial Page

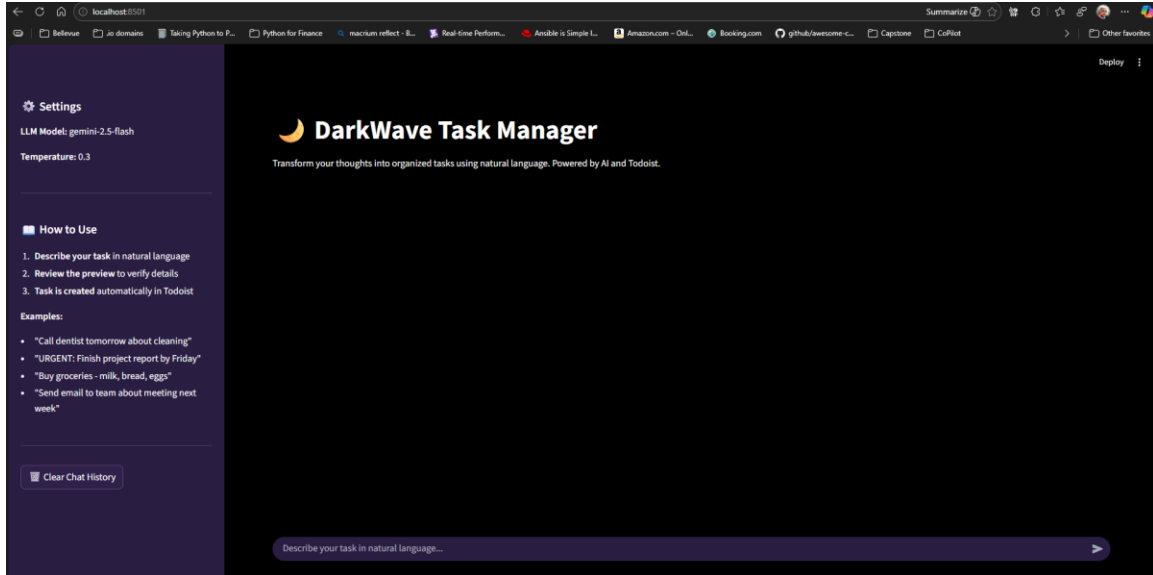


Figure 2: Simple Greeting – Chat Mode

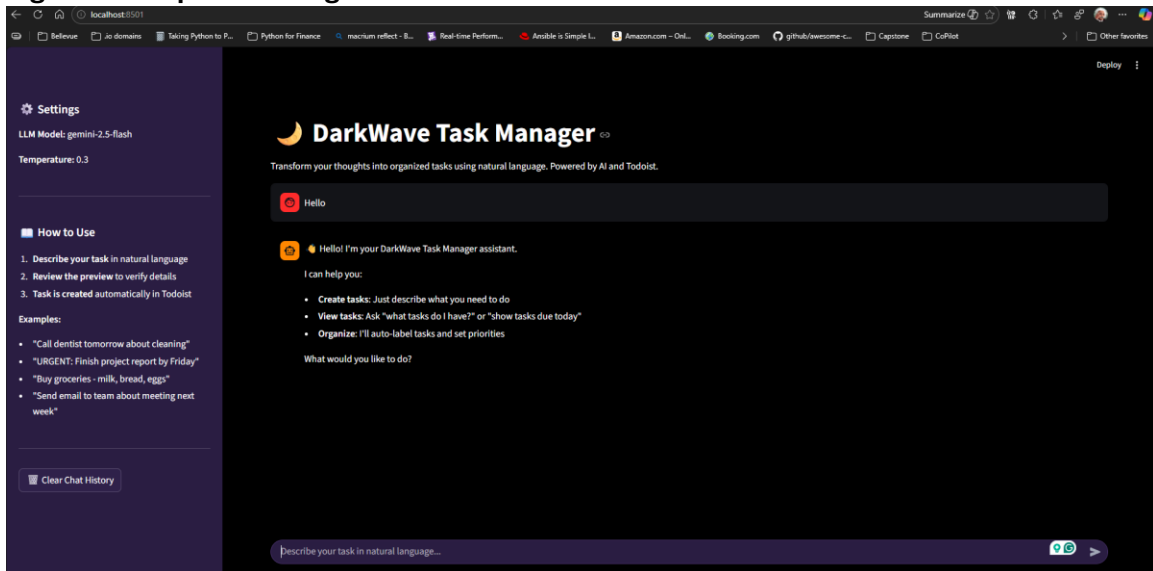


Figure 3: Show Task List

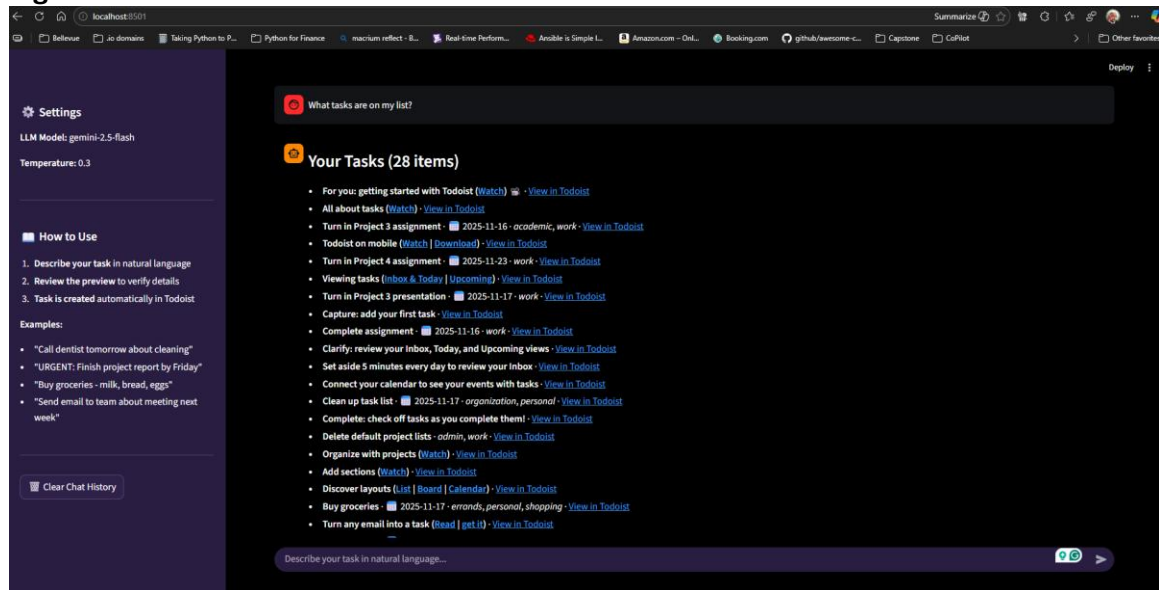


Figure 4: Adding a new item to the task list

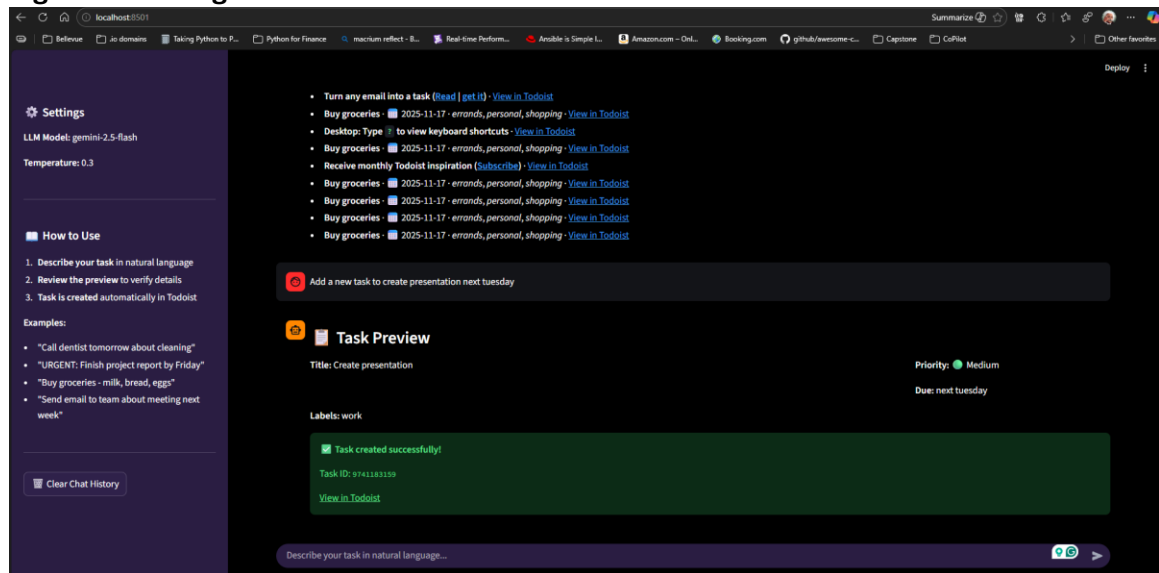


Figure 5: Thinking

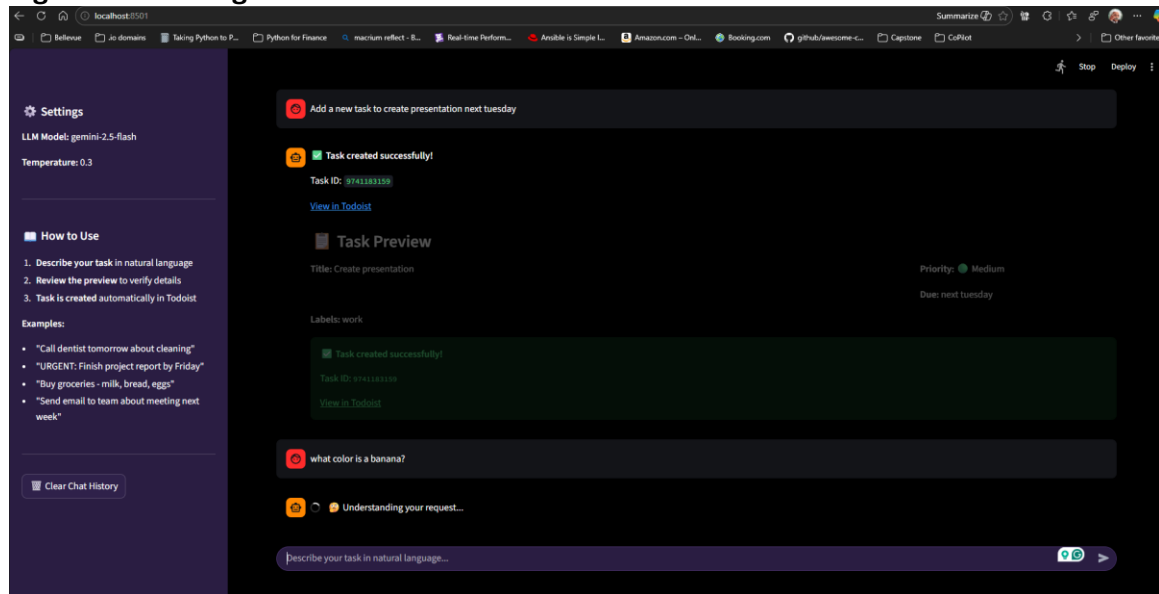
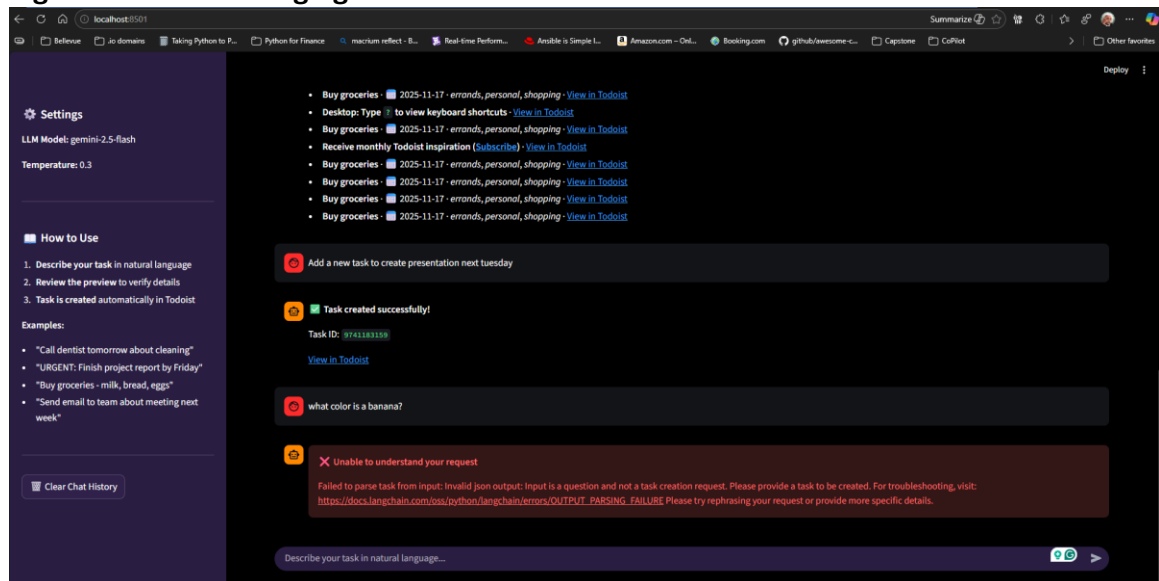


Figure 6: Error Messaging



Appendix B

Potential Audience Questions

1. How does the system actually work when a user types in a task?
2. What happens if the AI misunderstands what the user wants?
3. Why did you only build task creation and listing instead of all the features?
4. What is the difference between running it locally on your computer versus in a container?
5. How does the system actually work when a user types in a task?
6. What was the most challenging part of this project?
7. How do you keep user credentials and task data secure?
8. Did GitHub Copilot actually help, or did it make mistakes?
9. What would you need to add to make this ready for real users?
10. Could this same approach work with other apps besides Todoist, like Google Calendar or email?

References

- Astral. (2023). Ruff: An extremely fast Python linter and code formatter. <https://docs.astral.sh/ruff/>
- Astral. (2024). uv: A high-speed Python package and project manager. <https://docs.astral.sh/uv/>
- Colvin, S. (2024). Pydantic: Data validation and settings management using Python type annotations. <https://docs.pydantic.dev/>
- Doist. (n.d.). Todoist API documentation. <https://developer.todoist.com/>
- Furze, L. (2024, July 29). Creating a project assistant with Claude and Todoist. <https://leonfurze.com/2024/07/29/creating-a-project-assistant-with-claude-and-todoist/>
- GitHub. (2024). GitHub Copilot: Your AI pair programmer. <https://github.com/features/copilot>
- Google. (2024a). Gemini API: Built with Gemini models. <https://ai.google.dev/>
- Google. (2024b). Responsible AI practices for Gemini models. <https://ai.google.dev/responsible>
- Lehtosalo, J., et al. (2024). mypy: Optional static typing for Python. <http://mypy-lang.org/>
- LangChain. (2024). LangChain Python documentation. <https://python.langchain.com/>
- Lehtinen, A. (2024). pre-commit: A framework for managing and maintaining multi-language pre-commit hooks. <https://pre-commit.com/>
- PyCQA. (2024). Bandit: Security linter for Python source code. <https://bandit.readthedocs.io/>
- Red Hat. (2024). Podman: A tool for managing OCI containers and pods. <https://podman.io/>

- Streamlit Inc. (2023). Streamlit: The fastest way to build data apps.
<https://streamlit.io/>